DICT DATA SCIENCE SEMINAR · BAGUIO · DAY 1

Activity 2 — Web Scraping with Python

What you'll learn

Web scraping means pulling data straight off web pages with code. You'll do it two ways: first from a **live website** (Hacker News headlines), then from a **prepared data file** (the CAR-region dataset), and you'll turn both into tables and charts.

Before you start

- A **Google account** to use Google Colab.
- For Part B you'll need **car_data.html** — either upload it (Step 8) or download it with code (optional cell below).

> **Honest note:** the CAR dataset is **simulated sample data** made for this workshop. The numbers are realistic but invented — not official government figures. What's real is the **skill**.

Run each cell with **Shift + Enter**, top to bottom.

PART A — Scraping a Live Website (Hacker News)

Step 1 — Install the scraping tools

COPY THIS CODE

```
!pip install beautifulsoup4 requests
```

> **You should see:** install messages or "Requirement already satisfied".

Step 2 — Load the libraries

COPY THIS CODE

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
from IPython.display import display

print("Libraries loaded.")
```

> **You should see:** `Libraries loaded.`

Step 3 — Download the web page

`requests.get` fetches the raw page. A status of **200** means success.

COPY THIS CODE

```
url = 'https://news.ycombinator.com'

try:
    response = requests.get(url, timeout=10)
    print(f"Status code: {response.status_code}")
    if response.status_code == 200:
        print("Page downloaded successfully.")
    else:
        print("Got a response, but not 200. The site may be busy.")
except Exception as e:
    print(f"Could not reach the site: {e}")
    print("Check your internet connection and try again.")
```

> **You should see:** `Status code: 200` and "Page downloaded successfully."

Step 4 — Parse the HTML and grab the headlines

`BeautifulSoup` turns the raw HTML into something we can search. We pick out the headline links.

COPY THIS CODE

```
soup = BeautifulSoup(response.text, 'html.parser')
titles = soup.select('.titleline > a')

print(f"Found {len(titles)} headlines.")
print("\nFirst 3 as a preview:")
for t in titles[:3]:
    print(" -", t.text)
```

> **You should see:** "Found 30 headlines." (give or take) and a 3-headline preview.

Step 5 — Put the headlines into a table

COPY THIS CODE

```
rows = []
for rank, t in enumerate(titles, start=1):
    rows.append({'Rank': rank, 'Headline': t.text, 'Link': t.get('href', '')})

df_news = pd.DataFrame(rows)
print(f"Collected {len(df_news)} headlines into a table.")
display(df_news.head(10))
```

> **You should see:** a table with Rank, Headline, and Link columns.

Step 6 — How long are the headlines?

A first analysis: measure each headline's length in characters.

COPY THIS CODE

```
df_news['Length'] = df_news['Headline'].str.len()

print(f"Average length: {df_news['Length'].mean():.1f} characters")
print(f"Shortest: {df_news['Length'].min()} | Longest: {df_news['Length'].max()}")

plt.figure(figsize=(10, 5))
plt.hist(df_news['Length'], bins=15, color='skyblue', edgecolor='black', alpha=0.8)
plt.title('Distribution of Headline Lengths', fontweight='bold')
plt.xlabel('Characters')
plt.ylabel('Number of headlines')
plt.grid(axis='y', alpha=0.3)
plt.show()
```

> **You should see:** the average length and a histogram.

Step 7 — Which words show up most?

We split every headline into words, drop tiny filler words, and count the rest.

COPY THIS CODE

```
text = ' '.join(df_news['Headline']).lower()
stop_words = {'the','a','an','and','or','but','in','on','at','to','for','of',
              'with','by','is','are','was','were','from','this','that','as'}
words = [w.strip('.,:;!?()[]"\'') for w in text.split()]
words = [w for w in words if len(w) > 3 and w not in stop_words]

top = Counter(words).most_common(10)
words_df = pd.DataFrame(top, columns=['Word', 'Count'])

plt.figure(figsize=(10, 6))
bars = plt.barh(words_df['Word'], words_df['Count'], color='lightcoral')
plt.gca().invert_yaxis()  # most common on top
for bar, c in zip(bars, words_df['Count']):
    plt.text(bar.get_width() + 0.1, bar.get_y() + bar.get_height()/2,
             str(c), va='center', fontweight='bold')
plt.title('Most Common Words in Headlines Today', fontweight='bold')
plt.xlabel('Times used')
plt.tight_layout()
plt.show()
```

> **You should see:** a horizontal bar chart of the top 10 words.

PART B — Scraping the CAR Region Dataset

Step 8 — Get the data file into Colab

Click the **folder icon** on the left of Colab, then the **upload icon**, and upload **car_data.html**. Then run this cell to confirm Colab can see it.

COPY THIS CODE

```
import os
if os.path.exists('car_data.html'):
    print("Found car_data.html, you're ready.")
else:
    print("car_data.html not found yet.")
    print("Upload it using the folder icon on the left, then re-run this cell.")
```

> **Can't upload?** Easier alternative — run this cell to download the file directly into Colab, then run Step 8 to confirm:

DOWNLOAD INSTEAD OF UPLOAD

```
import urllib.request
urllib.request.urlretrieve(
    "https://datasciencebaguio-production.up.railway.app/car_data",
    "car_data.html")
print("Downloaded car_data.html")
```

> **You should see:** `Found car_data.html, you're ready.`

Step 9 — Scrape all three tables at once

`pd.read_html` reads every table on the page into a list. This page has three.

COPY THIS CODE

```
tables = pd.read_html('car_data.html')
print(f"Scraped {len(tables)} tables from the page.\n")

weather_df = tables[0]
schools_df = tables[1]
tourism_df = tables[2]

print(f"Weather stations: {len(weather_df)} rows")
print(f"Schools: {len(schools_df)} rows")
print(f"Tourism sites: {len(tourism_df)} rows")
```

> **You should see:** Weather **46**, Schools **50**, Tourism **40** rows.

Step 10 — Look at what we scraped

Always eyeball your data before analyzing it.

COPY THIS CODE

```
print("WEATHER STATIONS (first 5):")
display(weather_df.head())
print("SCHOOLS (first 5):")
display(schools_df.head())
print("TOURISM SITES (first 5):")
display(tourism_df.head())
```

> **You should see:** the first 5 rows of each table.

Step 11 — Quick summary numbers

COPY THIS CODE

```
print("WEATHER")
print(f"  Average temperature: {weather_df['Temperature_C'].mean():.1f} C")
print(f"  Average rainfall: {weather_df['Rainfall_mm'].mean():.1f} mm")

print("\nSCHOOLS")
print(f"  Total enrollment: {schools_df['Enrollment'].sum():,} students")
print(f"  Average class ratio: {schools_df['Student_Teacher_Ratio'].mean():.1f} students
per teacher")

print("\nTOURISM")
print(f"  Total annual visitors: {tourism_df['Annual_Visitors'].sum():,}")
print(f"  Average rating: {tourism_df['Rating'].mean():.2f} out of 5")
```

> **You should see:** headline figures for each table (e.g. avg temperature ~24 C).

Step 12 — The analysis dashboard

Six charts in one figure, pulling from all three scraped tables.

COPY THIS CODE

```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('CAR Region Data Dashboard (Simulated Sample Data)', fontsize=15,
fontweight='bold')

# 1. Temperature by province
weather_df.boxplot(column='Temperature_C', by='Province', ax=axes[0, 0])
axes[0, 0].set_title('Temperature by Province')
axes[0, 0].set_xlabel('')
axes[0, 0].tick_params(axis='x', rotation=45)

# 2. Students by school type
by_type = schools_df.groupby('Type')['Enrollment'].sum()
axes[0, 1].pie(by_type.values, labels=by_type.index, autopct='%1.0f%%', startangle=90)
axes[0, 1].set_title('Students by School Type')

# 3. Rainfall vs elevation
axes[0, 2].scatter(weather_df['Elevation_m'], weather_df['Rainfall_mm'], alpha=0.6,
color='teal')
axes[0, 2].set_title('Rainfall vs Elevation')
axes[0, 2].set_xlabel('Elevation (m)')
axes[0, 2].set_ylabel('Rainfall (mm)')

# 4. Tourism ratings
axes[1, 0].hist(tourism_df['Rating'], bins=12, color='gold', edgecolor='black')
axes[1, 0].axvline(tourism_df['Rating'].mean(), color='red', linestyle='--',
                   label=f"Mean {tourism_df['Rating'].mean():.2f}")
axes[1, 0].set_title('Tourism Site Ratings')
axes[1, 0].set_xlabel('Rating')
axes[1, 0].legend()

# 5. Visitors by tourism type
by_t = tourism_df.groupby('Type')['Annual_Visitors'].sum().sort_values()
axes[1, 1].barh(by_t.index, by_t.values, color='coral')
axes[1, 1].set_title('Annual Visitors by Type')
axes[1, 1].set_xlabel('Visitors')

# 6. Weather conditions
cond = weather_df['Condition'].value_counts()
axes[1, 2].bar(range(len(cond)), cond.values, color='lightblue')
axes[1, 2].set_xticks(range(len(cond)))
axes[1, 2].set_xticklabels(cond.index, rotation=45, ha='right')
axes[1, 2].set_title('Weather Conditions')
```

```
axes[1, 2].set_ylabel('Stations')

plt.tight_layout()
plt.show()
print("Dashboard built from the scraped CAR data.")
```

> **You should see:** a 6-chart dashboard (boxplot, pie, scatter, histogram, bar, bar).

Tips & common errors

- Run cells **top to bottom** with Shift + Enter.
- **Step 4 found 0 headlines?** The live site's layout may have changed — re-run, or try again later.
- **FileNotFoundError in Step 9?** car_data.html isn't loaded — do Step 8 (upload or the optional download cell) first.
- Always check a site's **terms** before scraping it, and always be clear whether data is **real or simulated**.

What's next

In **Activity 3** you'll take this scraped tourism data and run a full **exploratory data analysis**.
